

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

PYTHON LAB PROJECT REPORT

TITLE : Intelligent Disaster Data Interoperability and Emergency Response Coordination Platform

BY TEAM

SALVA SOUMYA , 23MID0061

GILLEDI SANTHOSH , 23MID023

UNDER THE GUIDANCE OF

Dr.SHARMILA BANU K

Member 1: Disaster Data Integration and Emergency Response Coordination System (23MID0061)

Member 2 : Intelligent Disaster Assessment and Resource Management System (23MID0233)

TITLE : Intelligent Disaster Data Interoperability and Emergency Response Coordination Platform

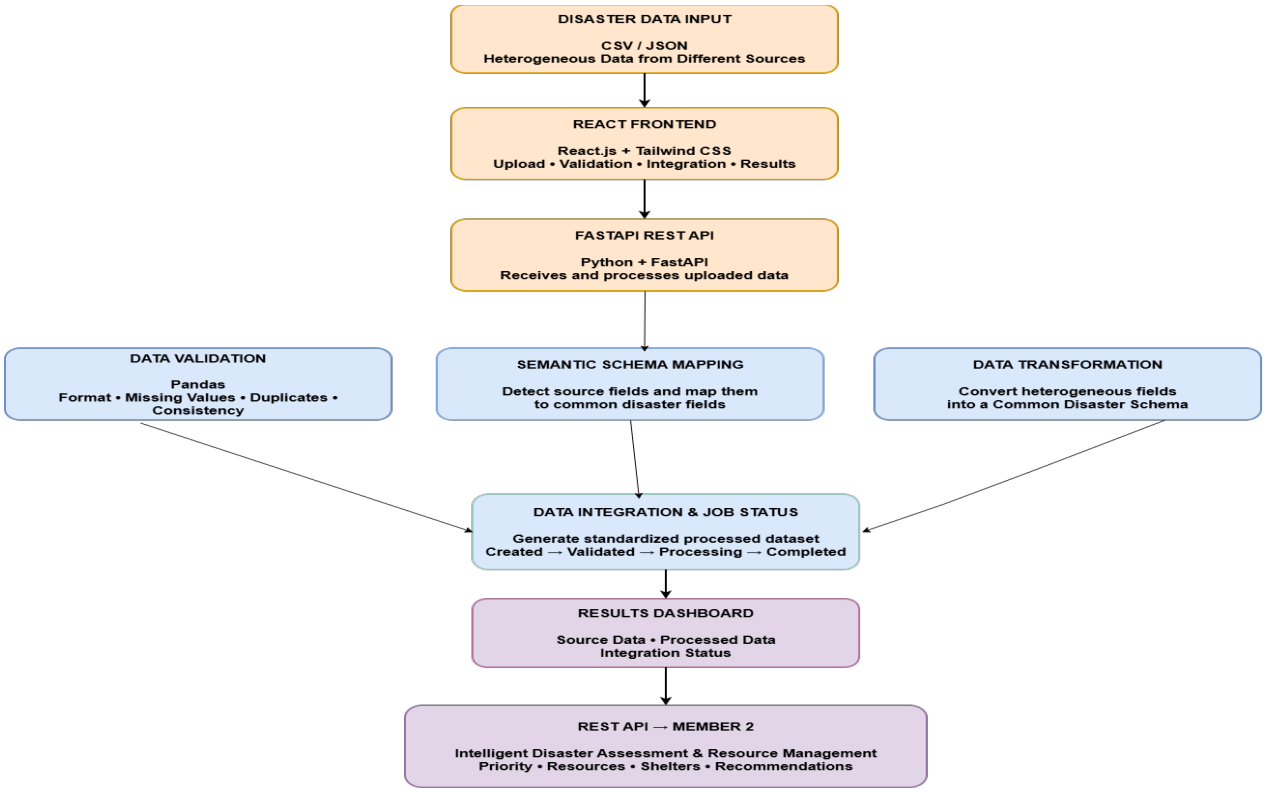
PROJECT DESCRIPTION :

InteroperAI-ResQUE is an intelligent disaster-response platform designed to overcome the lack of interoperability between different disaster management systems. It accepts heterogeneous disaster data from multiple sources, validates and transforms it into a common standardized structure, and enables seamless data exchange through APIs. The integrated data is then analyzed to assess disaster priority, resource requirements, shelter capacity, and affected populations. Based on this analysis, the system provides actionable recommendations to support faster, consistent, and coordinated emergency-response decision making.

MY CONTRIBUTION :

Disaster Data Integration and Emergency Response Coordination System

SYSTEM ARCHITECTURE :



OBJECTIVES :

1. Heterogeneous Data Collection

To provide a web interface for uploading disaster datasets from different sources in CSV and JSON formats**.**

2.Data Validation

To automatically validate uploaded datasets by checking:

- File format
- Required fields
- Missing values
- Duplicate records
- Data consistency
- Dataset structure

3.Semantic Schema Mapping

To identify equivalent fields even when different disaster systems use different names.

For example:

event_id → incident_identifier
disaster → disaster_type
area → incident_location
risk_level → priority_level
people_affected → affected_population
casualty_count → reported_casualties
event_time → incident_timestamp
response_state → response_status

4.Data Transformation

To transform the heterogeneous source data into a common disaster-response schema so that different systems can understand and exchange the information consistently.

5. Data Integration

To process and store the transformed dataset as an integrated output that can be consumed by downstream disaster-intelligence services.

6. Job Tracking

To track the processing lifecycle of each uploaded dataset:

Created → Validated → Processing → Completed

7.API-Based Interoperability

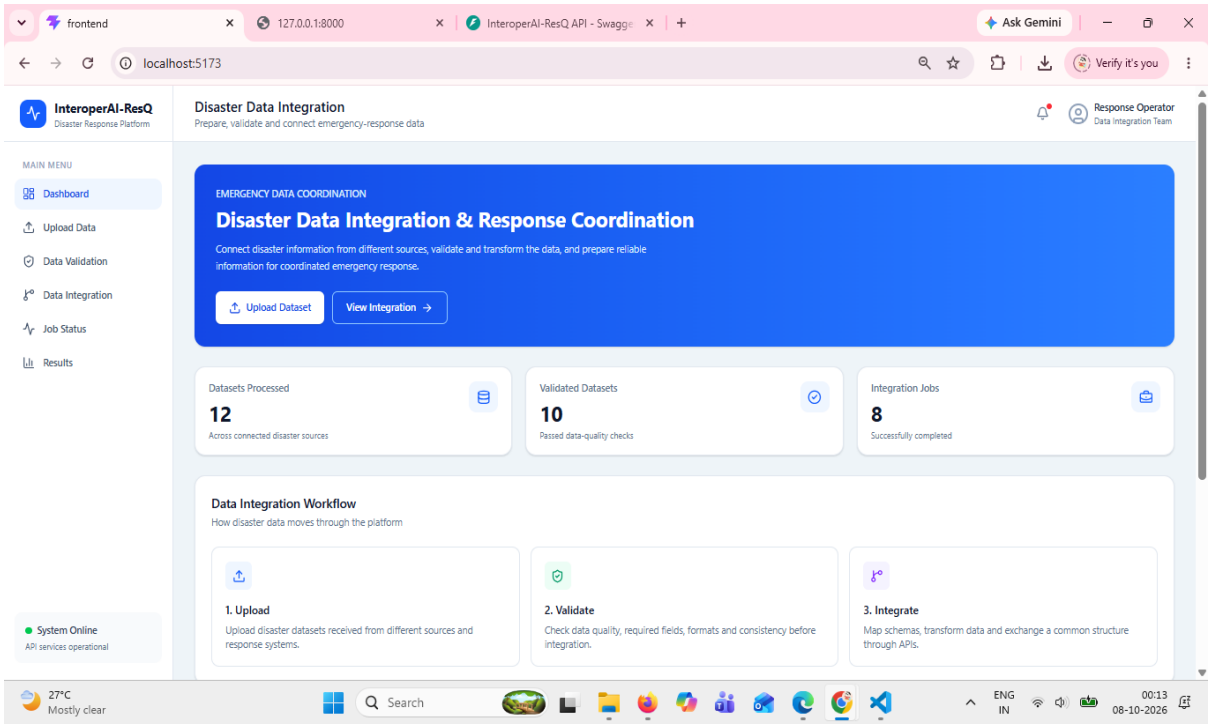
To expose standardized disaster data through REST APIs, allowing Member 2's intelligent assessment system to receive and analyze the integrated information.

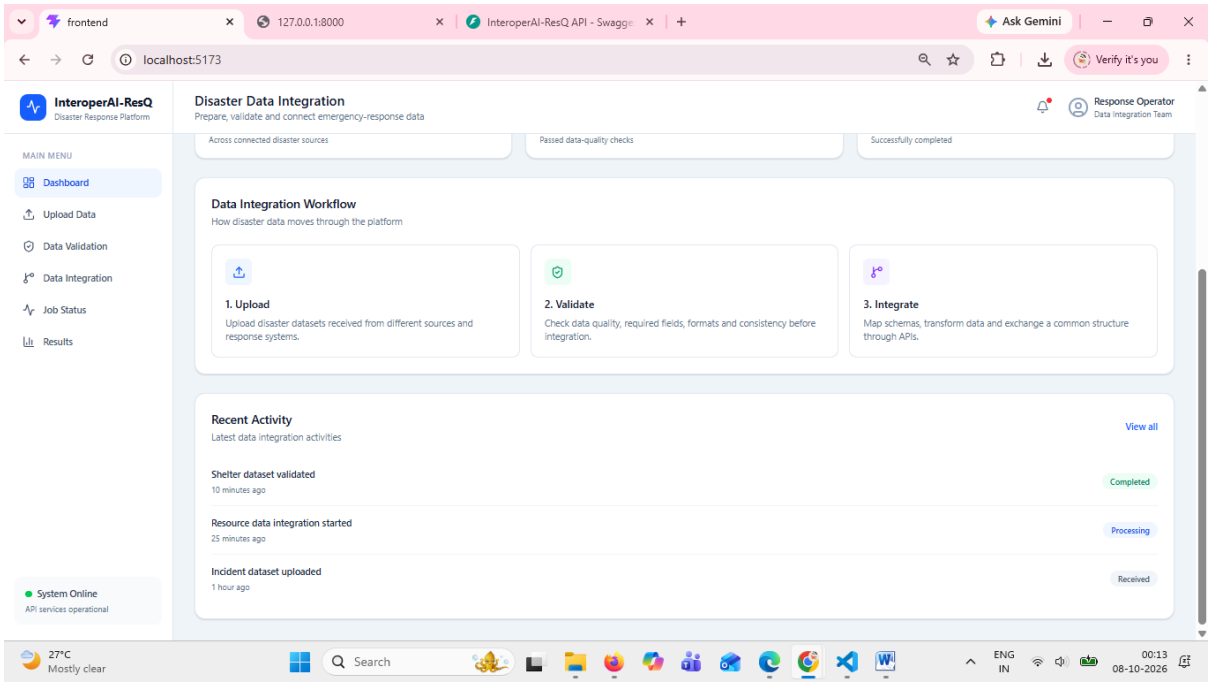
FRONTEND(http://localhost:5173)

Technologies Used: React.js, Tailwind CSS, React Router, Axios and Lucide React.

1.Dashboard

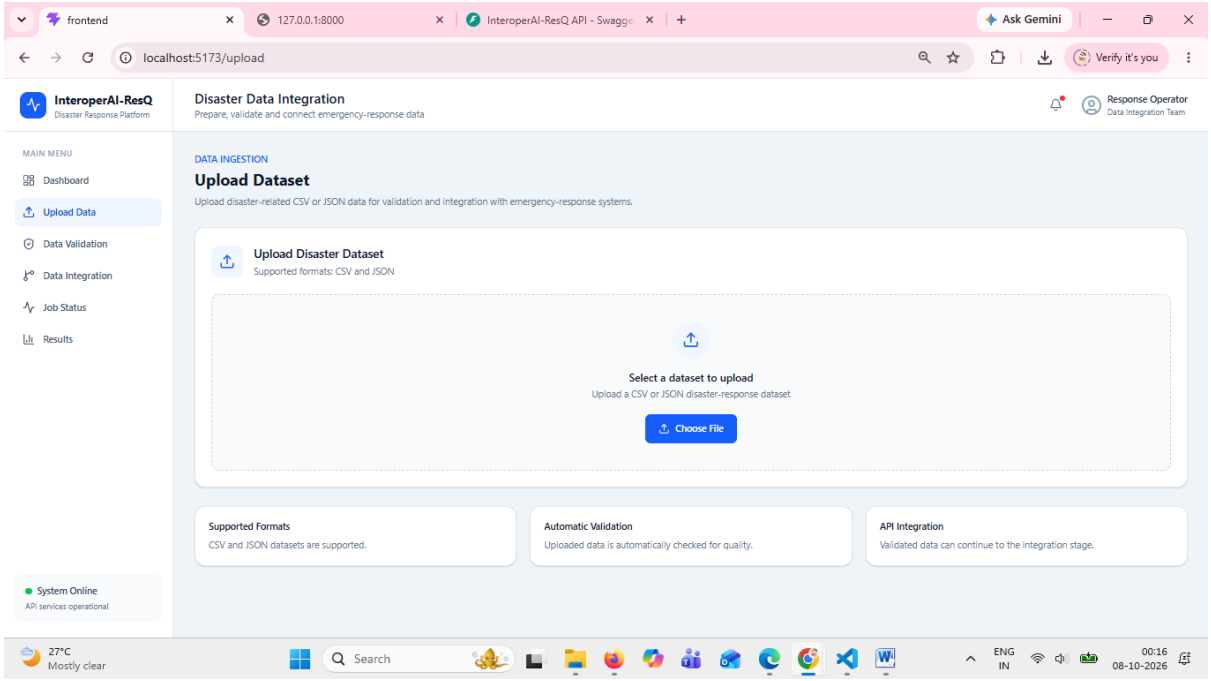
The main dashboard displays the primary data processing workflow across upload, validation, and integration stages. It includes high-level system indicators and recent activity logs to monitor dataset handling in real time.

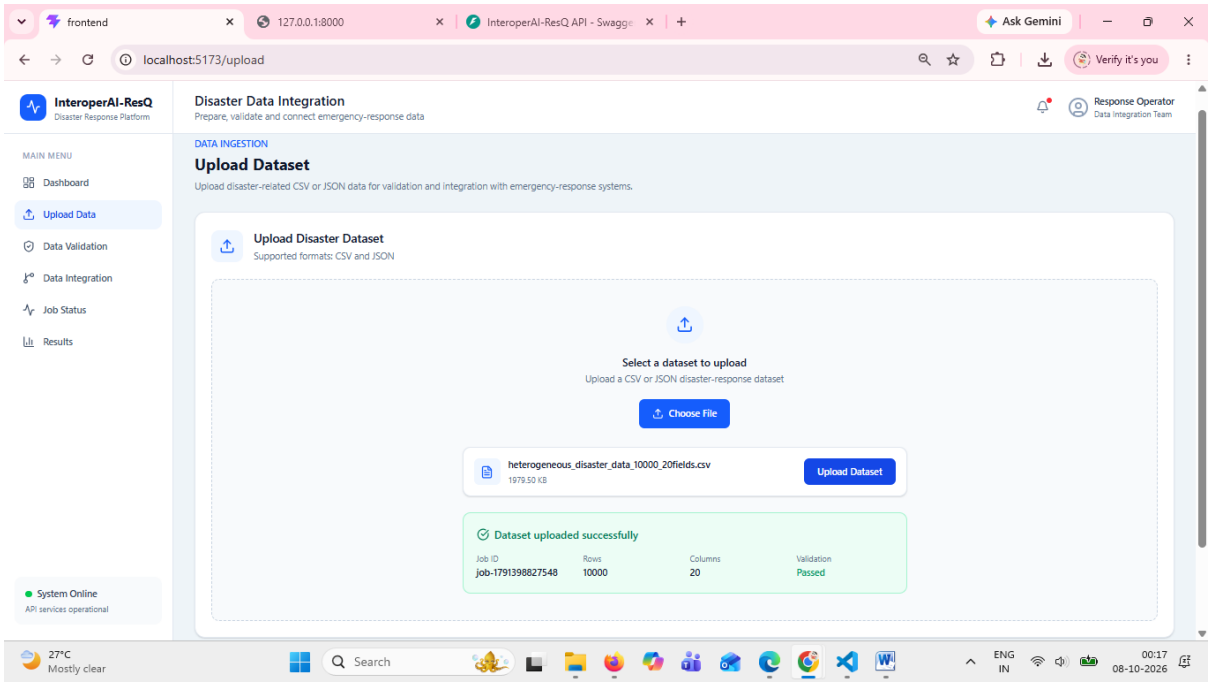




2. Upload data

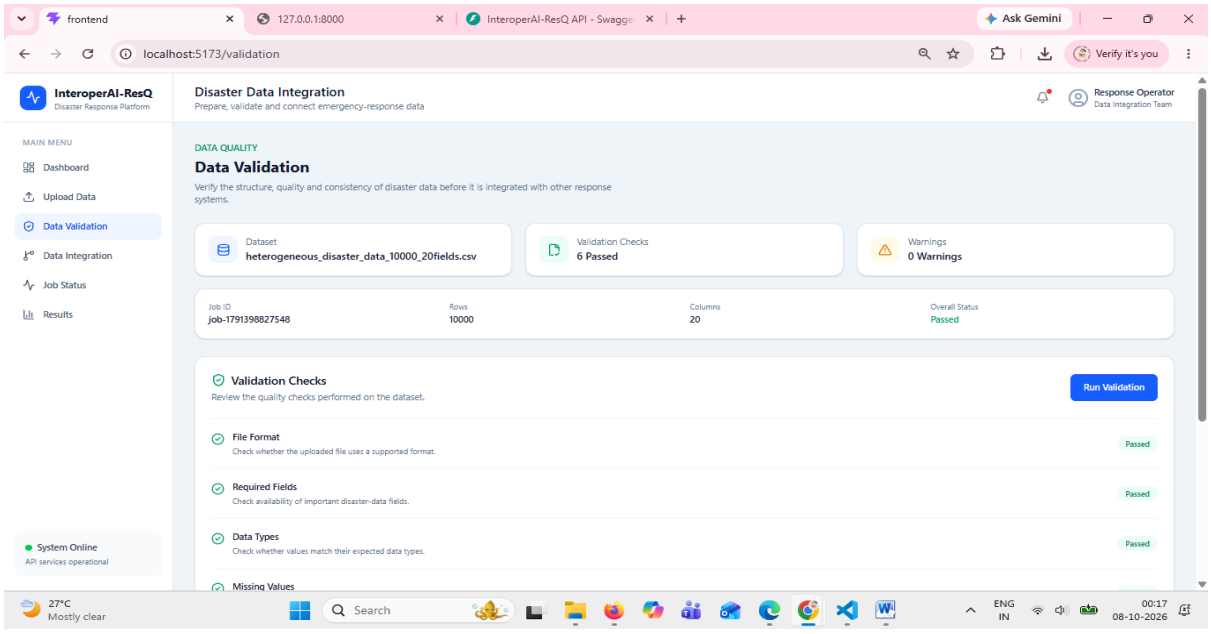
The upload interface allows users to select and ingest disaster datasets in CSV or JSON formats. Once a file is selected, it displays file details like total rows, column counts, and a generated job ID.

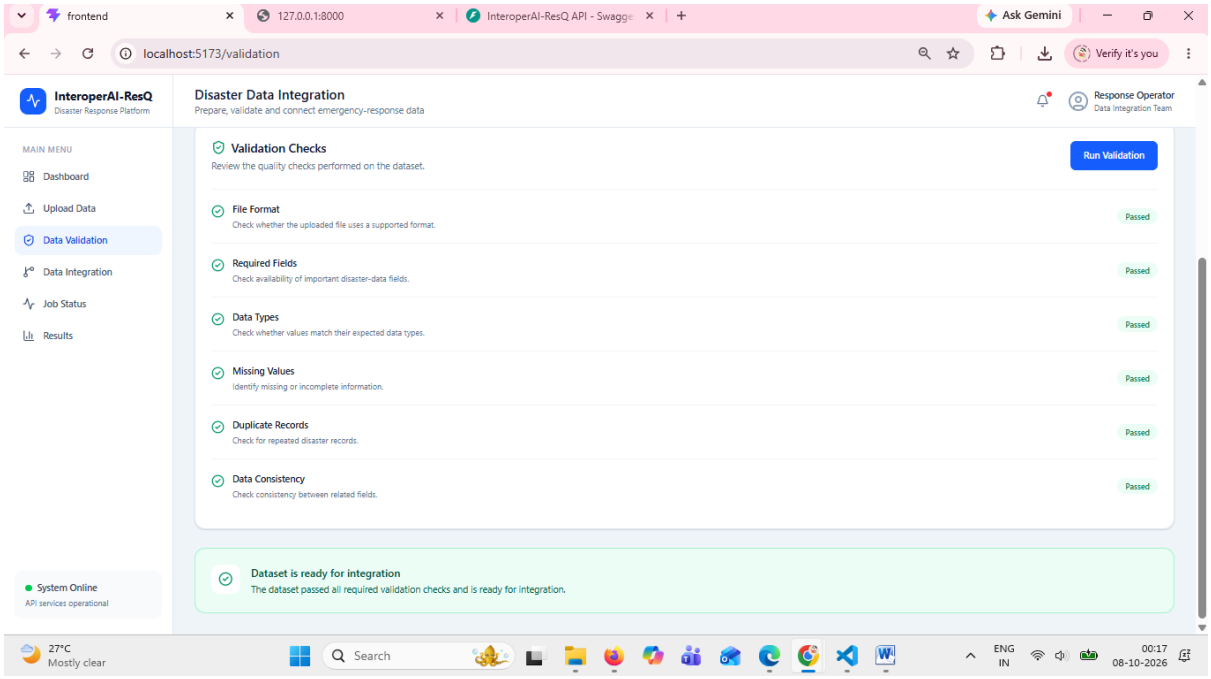




3.Data validaton

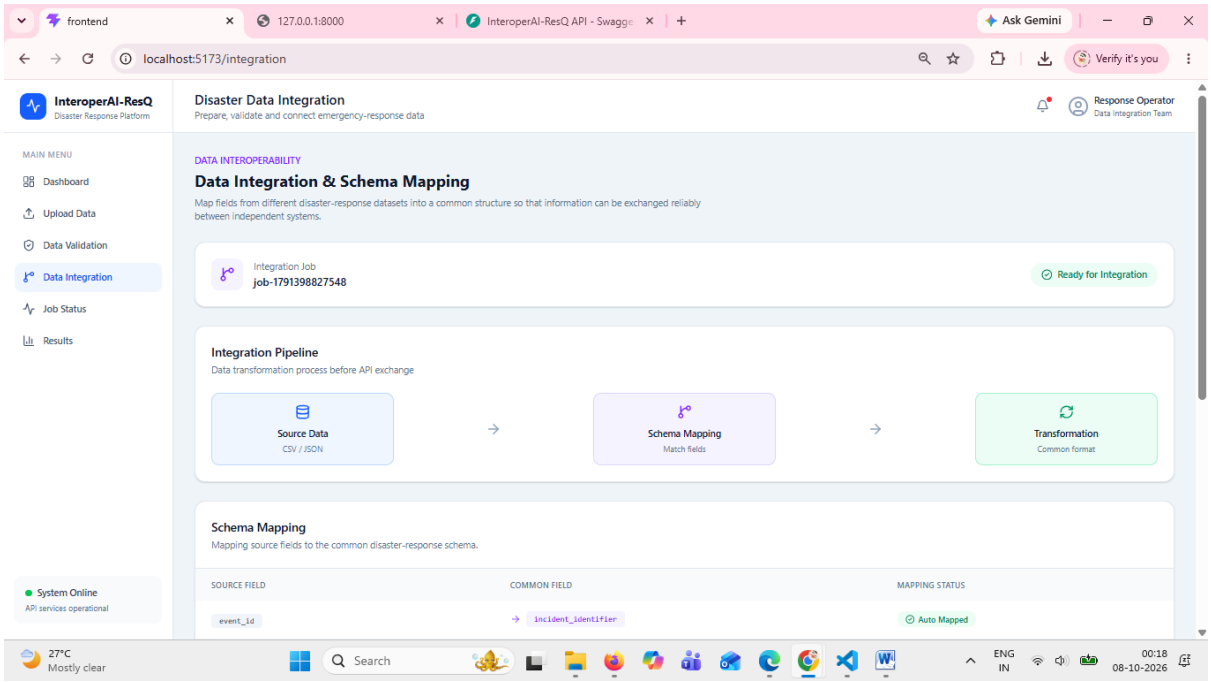
This view presents automated quality checks covering file format, required fields, data types, missing values, duplicates, and consistency. Status badges provide quick visual feedback confirming whether the dataset passes checks for integration.

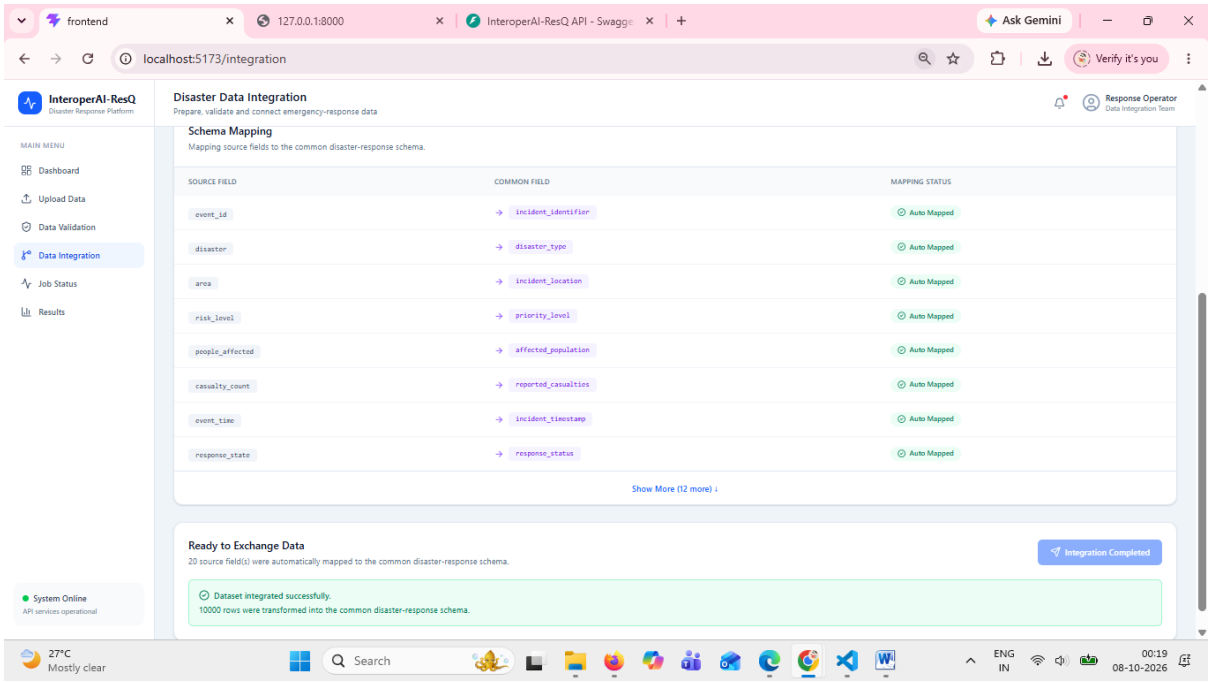




4.Data integration

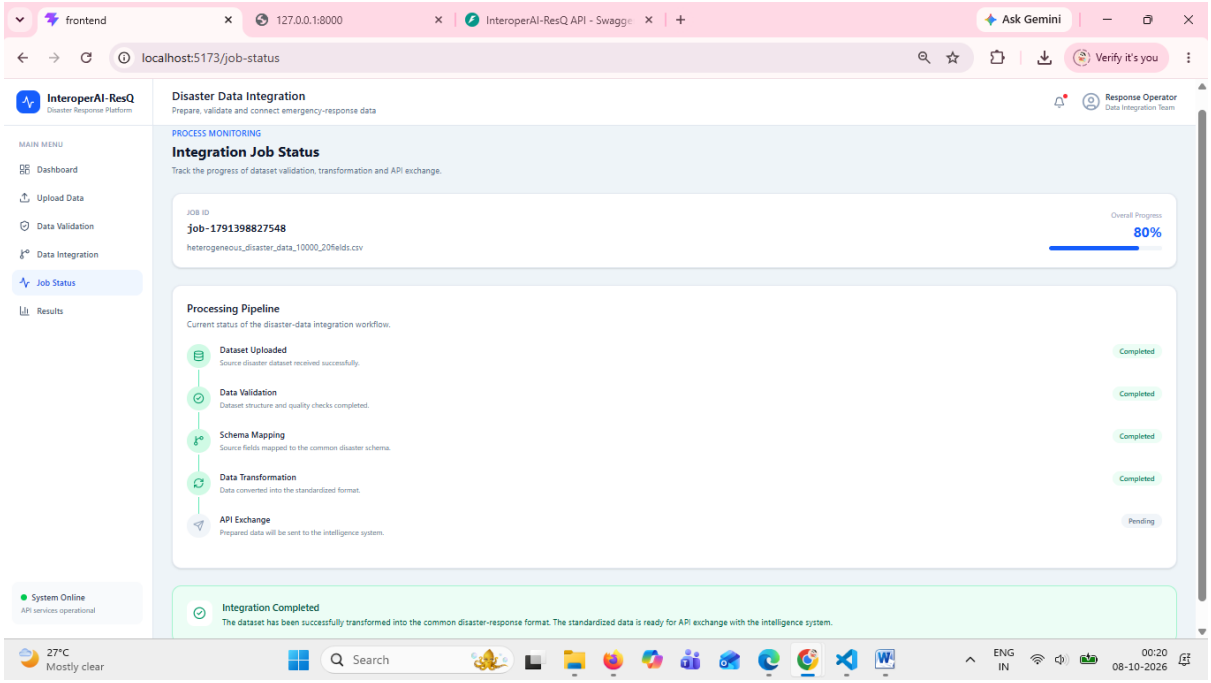
This interface maps source dataset column names to standardized emergency response schema fields. It illustrates the step-by-step pipeline from raw input fields to mapped and transformed outputs.





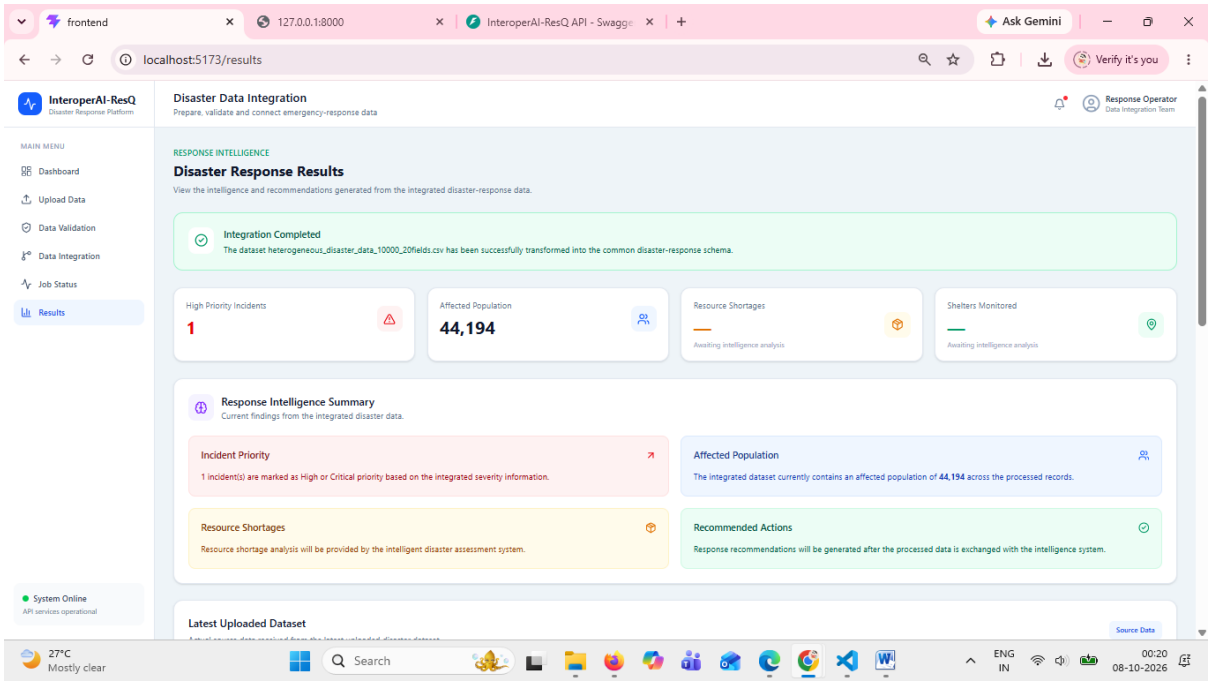
5.JOB STATUS

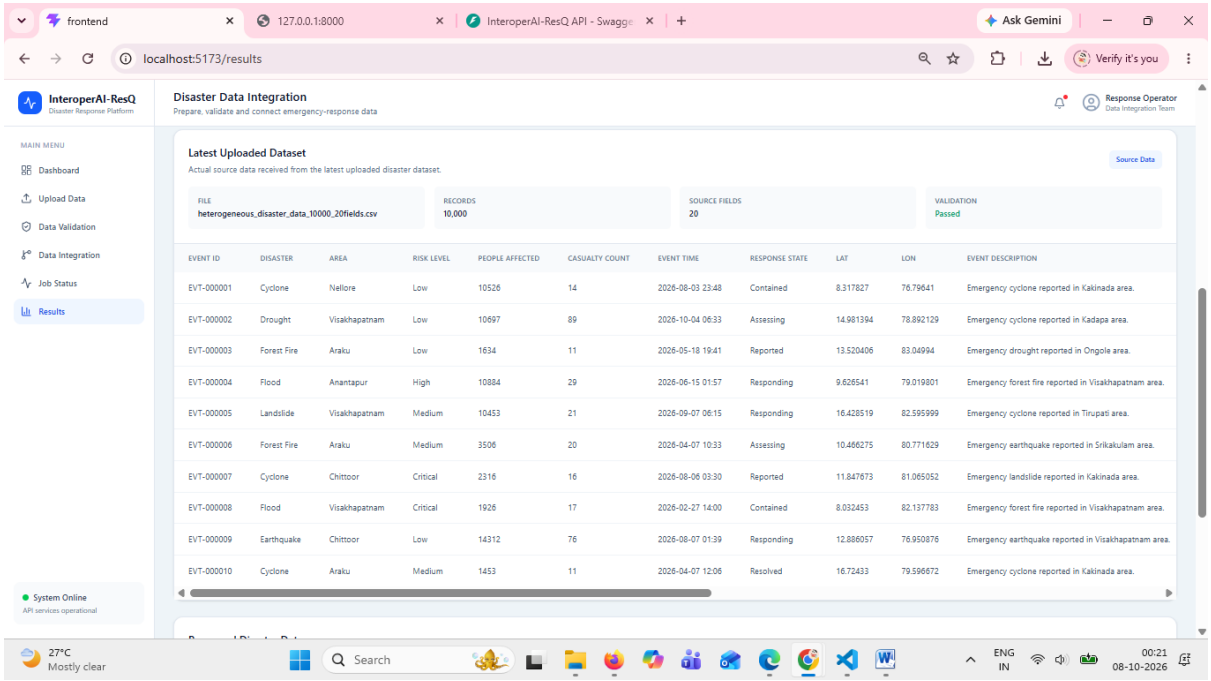
The job status view tracks real-time processing progress using a visual percentage bar and pipeline steps. It shows the completion status of each stage, from dataset upload through API exchange.

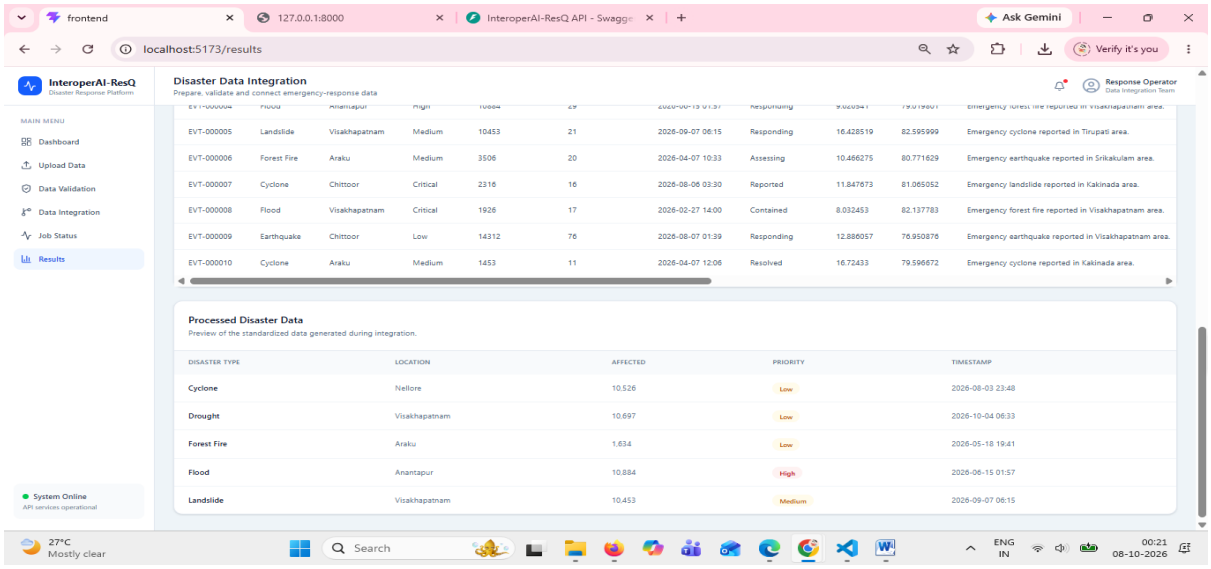


6.RESULTS :

The results view presents key emergency metrics including incident priority, affected population counts, and resource shortages. It also features tabular views displaying raw source data alongside the final standardized output.







BACKEND ARCHITECTURE:

Technologies Used: Python, FastAPI, Pandas, Pydantic, Uvicorn and REST APIs.

1.Backend – API Overview

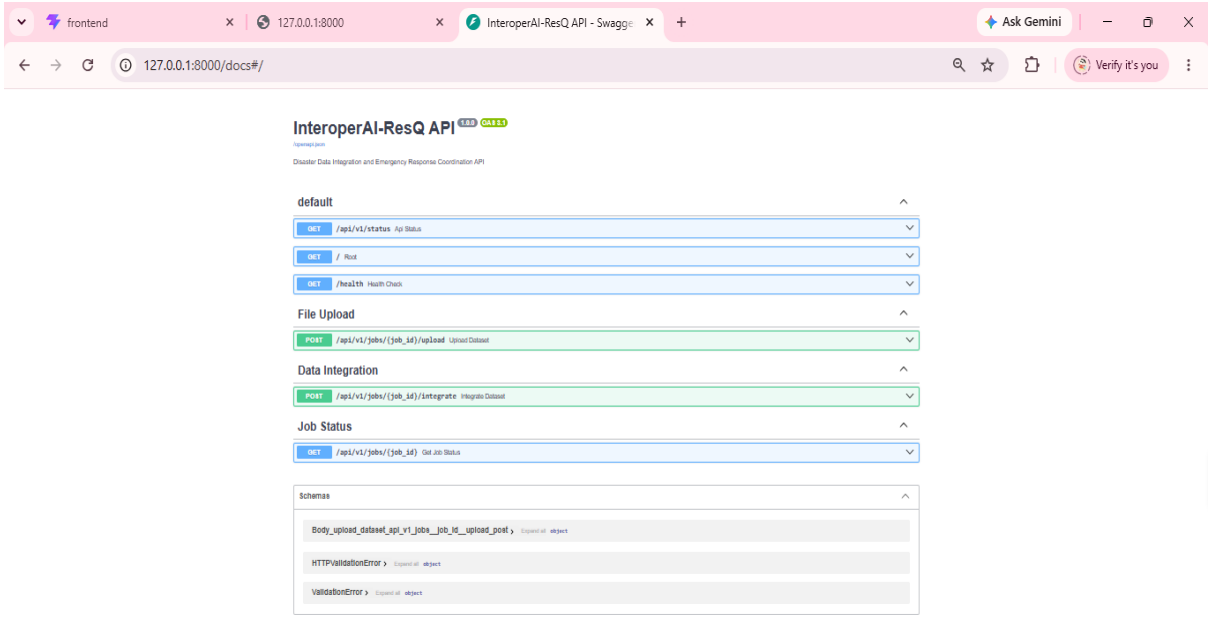
Main endpoints:

1.GET /api/v1/status — Checks API status

2.POST /api/v1/jobs/{job_id}/upload — Uploads and validates the dataset

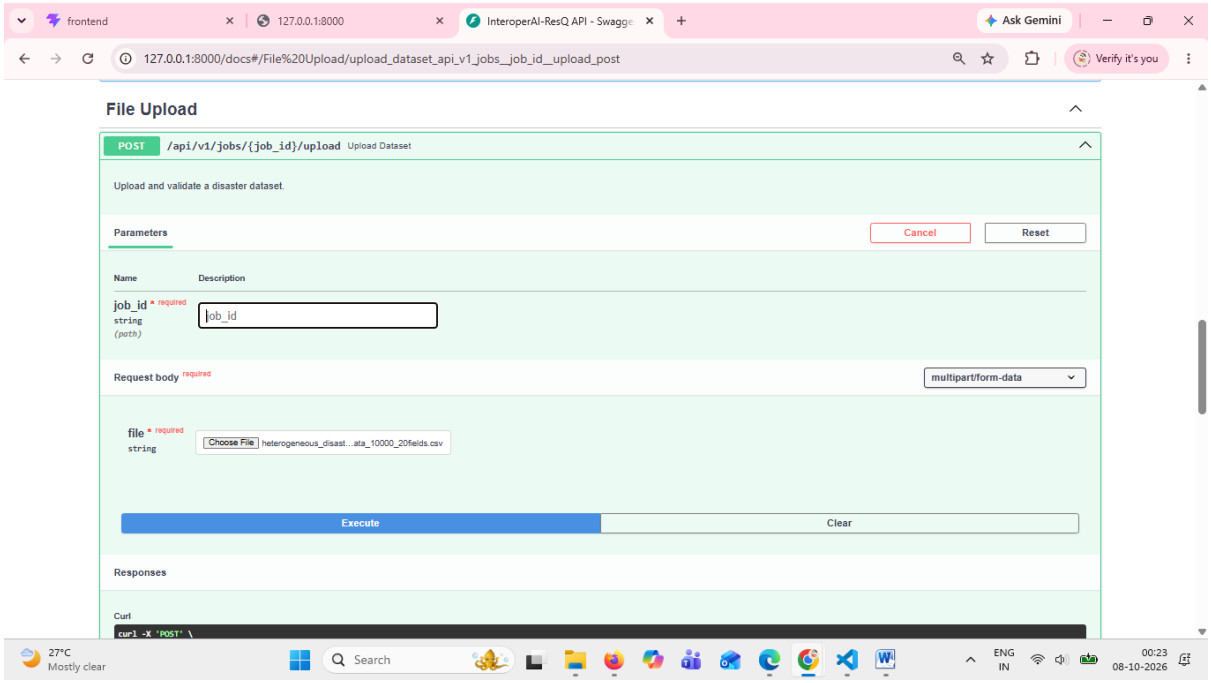
3.POST /api/v1/jobs/{job_id}/integrate — Integrates and transforms the dataset

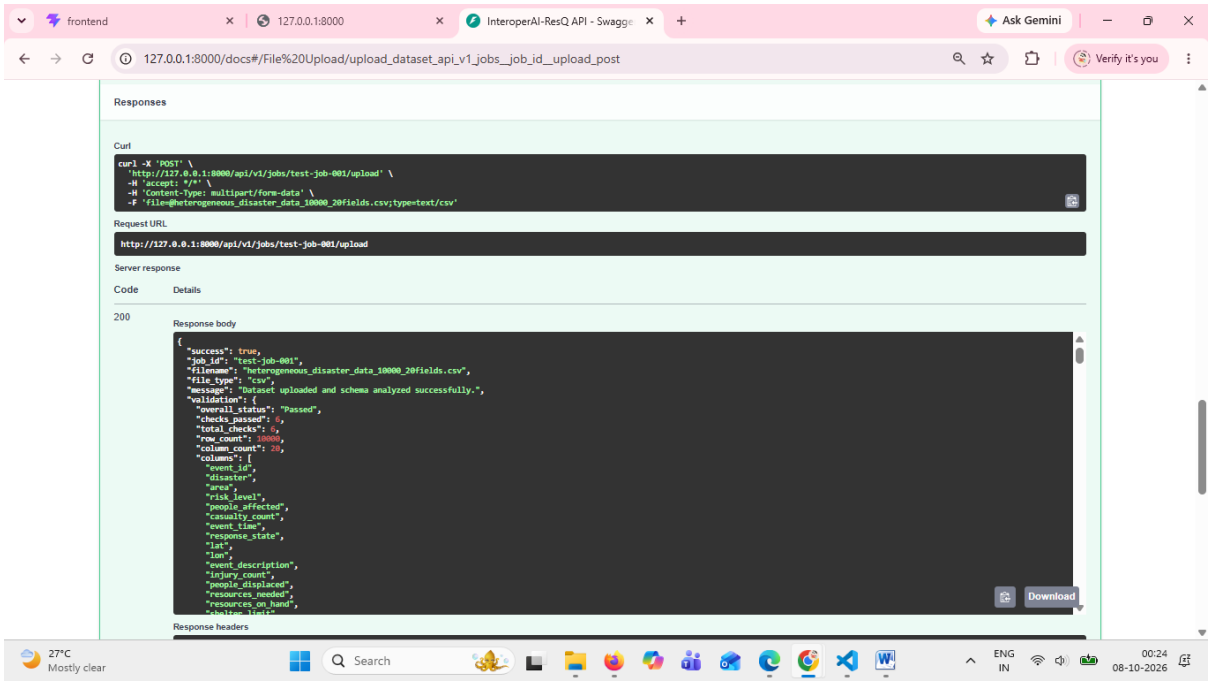
4.GET /api/v1/jobs/{job_id} — Retrieves job status



2.Dataset Upload API

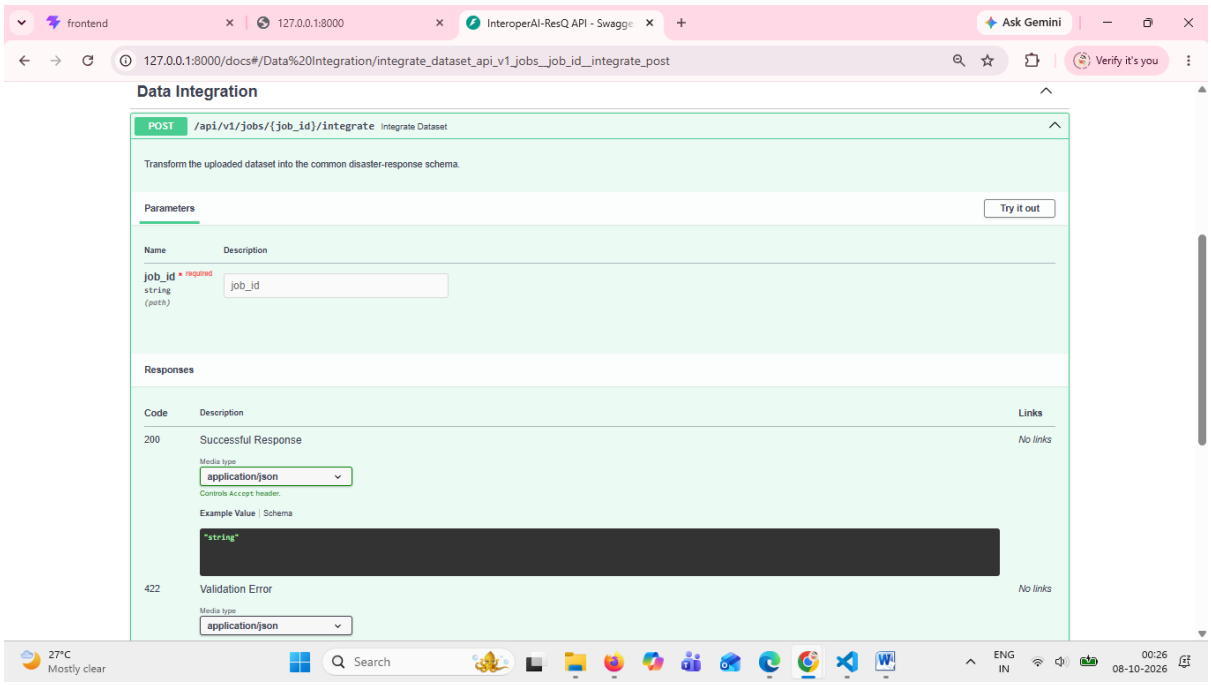
POST /api/v1/jobs/{job_id}/upload is used to upload the CSV/JSON disaster dataset to the backend.The backend reads and validates the uploaded data and returns the processing details.





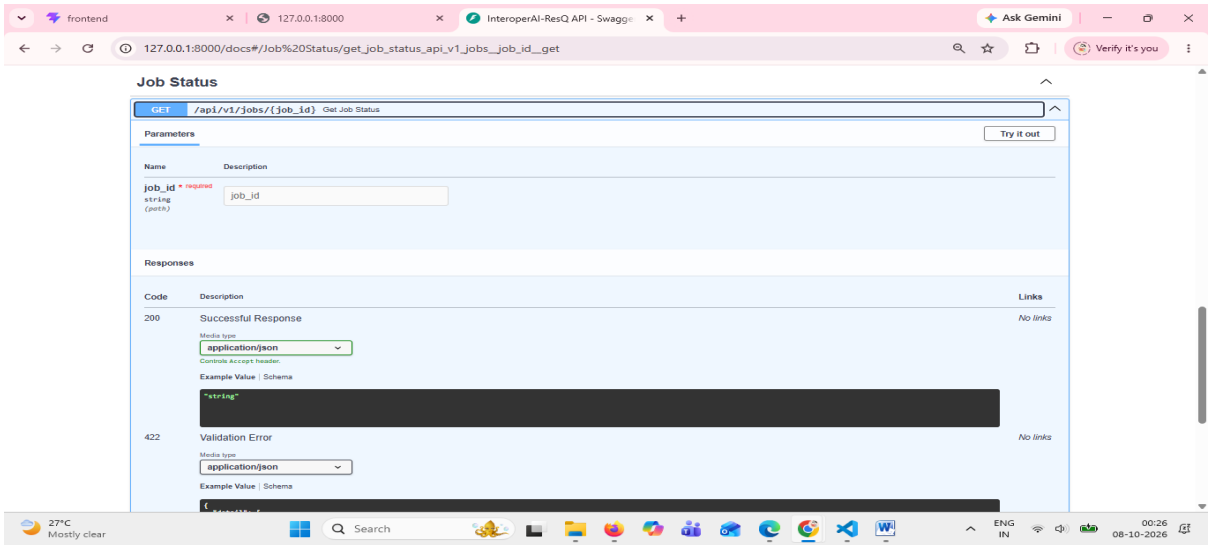
3.Data Integration API

POST /api/v1/jobs/{job_id}/integrate starts the data transformation and integration process.It converts the uploaded heterogeneous data into the common disaster-response structure.

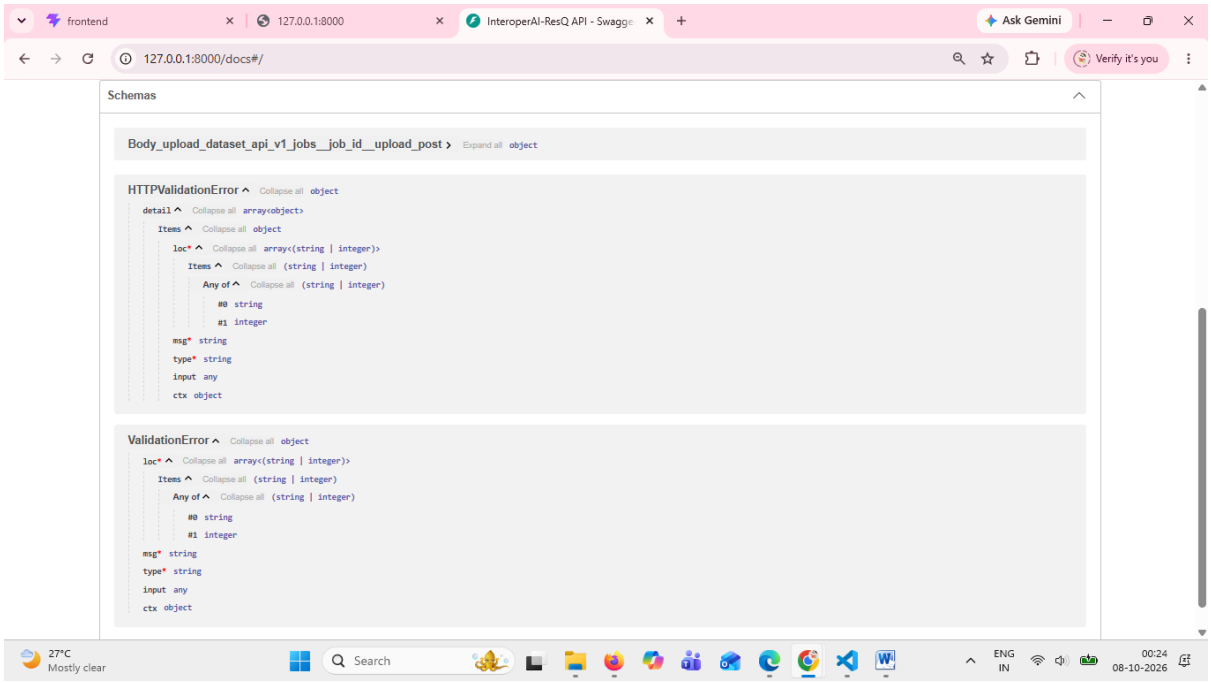


4.Job Status API

GET /api/v1/jobs/{job_id} retrieves the current status of the dataset processing job.It allows the frontend to display whether the integration is completed or still processing.



5.The API schemas define the expected data format for dataset uploads and provide structured validation error handling during processing



Hosting

The website is currently hosted and tested in a local development environment.

The React frontend runs using the Vite development server, while the FastAPI backend runs locally using Uvicorn.

Frontend: http://localhost:5173

Backend: http://127.0.0.1:8000

API Documentation: FastAPI Swagger UI at /docs

Cloud deployment is planned for a future phase using **Vercel for the frontend and Render for the backend.**

THANK YOU